

The Physics of an LLM Request

Inference Fundamentals, Part 1B: why a language model's speed comes down to two numbers on a spec sheet

Part 1A explained what inference is, why it became an infrastructure discipline, and the latency-throughput-cost triangle every decision lives in. This part explains why the triangle has the shape it does: what happens when a request hits a model, where the memory goes, and why batching works until it suddenly does not. Every number here is one you can derive yourself with a pen, and by the end you will be able to.

Where does the time go? For more than a decade that was my question, asked of large data platforms on behalf of enterprises, and it was always the same question whatever the vendor. Was this query reading data, computing, or waiting for a slot? Is the warehouse sized for the concurrency it actually sees, or for the concurrency someone guessed? Is the cache hitting, and for which queries? Why does the 95th percentile blow up at nine in the morning when the median is fine? And what does all of this cost per query, because someone in finance is going to ask.

The levers were familiar too. Prune partitions so the query reads less. Cluster the data so the reads are sequential. Serve repeated queries from the result cache instead of recomputing them. Size the compute for the workload and let it scale out when concurrency spikes. Suspend it when idle, and accept that the first query after a resume pays for a cold cache. Isolate the dashboard users from the overnight loads so one cannot sink the other. Performance work on a data platform is the discipline of knowing which of those levers a given symptom calls for, and being able to prove it with a number.

More recently the question has moved to language models: agentic systems, and the open-source and frontier models beneath them. I came to inference expecting to find a new set of rules. What I found instead was a system with two limits, memory bandwidth and arithmetic, and a small number of formulas that connect them to everything a user experiences. The levers even map. A model's weights are the table that every query has to scan in full. Batching requests is the same trade as raising concurrency on a warehouse: more work per pass, more waiting per request. The KV cache is the working set that decides how many queries fit in memory at once. Prefix caching is the result cache. Scale-to-zero is auto-suspend, cold start included. And the 95th-percentile spike at nine in the morning has exactly the same cause it always had: arrivals outrunning capacity.

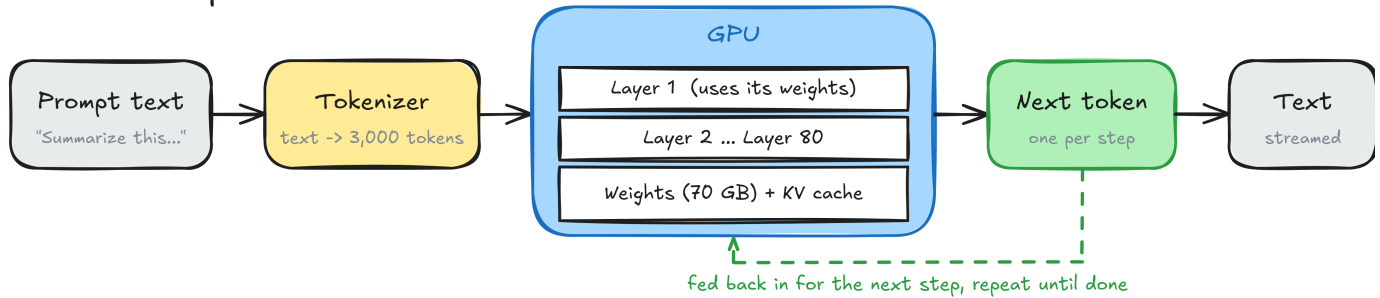
Once you can see the two bottlenecks, the behavior of these systems stops being mysterious. A benchmark table becomes something you can rebuild from first principles. An autoscaling setting becomes a statement about physics. That is what this series is for.

This part builds the physics up from the hardware. It is the longest article in the series, and it is the one that stands on its own.

Start with the picture

When a user sends a prompt to a model and gets an answer back, four things happen in sequence. The text is broken into tokens. The tokens are pushed through the model's layers, which use the model's weights. The model produces one new token at a time, each depending on the last. The tokens are turned back into text and streamed to the user.

What one request does



Every output token is a full trip through all the layers. A 300-token answer is 300 trips.

Every performance question in this field is about one of those four stages, and the rest of this article walks through them. Part 1A gave the two-ops-per-weight rule as a headline; here it is derived, along with everything that follows from it.

Tokens: the unit everything is measured in

A model does not read words. Before any text reaches it, a component called the tokenizer chops the text into tokens, which are chunks drawn from a fixed vocabulary of roughly 100,000 to 200,000 entries (Llama 3 uses 128,256). Common words are usually one token; rarer words, names, numbers and code are split into several. A workable rule for English prose, and the one OpenAI publishes, is that one token is about three quarters of a word or about four characters, so a thousand tokens is roughly 750 words, about two pages of text. Code and non-English languages tokenize less efficiently and can cost twice as many tokens for the same content.

The reason this matters is that every cost in inference is per token. The time to process a prompt is proportional to its tokens. The time to generate an answer is proportional to its tokens. The memory a request occupies is proportional to its tokens. Providers bill per token. So the first question about any workload is always the same: how many tokens go in, how many come out, and how much do those numbers vary from request to request. A workload that averages 2,000 input tokens but occasionally sends 30,000 is, for those occasional requests, a completely different workload.

Weights: the model itself

A model is, physically, an enormous list of numbers called parameters or weights. When people say "a 70B model" they mean seventy billion of these numbers. They were set during training and never change during inference; inference only reads them.

The weights have to live somewhere the GPU can read them quickly, and that place is the GPU's own memory, called HBM (high-bandwidth memory). Two properties of that memory drive almost everything in this series. Its **capacity** decides whether a model fits at all and how much room is left for the work of serving requests. Its **bandwidth**, meaning how many bytes per second can be read from it, decides how fast the model can generate tokens. An H100 has 80 gigabytes of capacity and about 3.35 terabytes per second of bandwidth. Hold on to both numbers.

Layers and the forward pass

Processing a token means running it through the model's layers in order. A 70B model has 80 layers; an 8B model has 32. A layer is the repeating unit a transformer is built from: think of the model as a stack of identical processing stages, and a token entering at the bottom passes through every stage before the model commits to its next token.

Each layer does two kinds of work. The first is a set of large matrix multiplications that combine the token's current representation with that layer's weights; this is where most of the arithmetic happens. The second is attention, in which the token looks back at every earlier token in the conversation and decides which ones matter for predicting what comes next. Running all the layers once is called a forward pass.

The amount of arithmetic in a forward pass is fixed by the model's size, and the rule is worth understanding rather than memorizing:

operations per token $\approx 2 \times$ number of parameters

70B model: $2 \times 70,000,000,000 = 140$ GFLOP per token

8B model: $2 \times 8,000,000,000 = 16$ GFLOP per token

A GFLOP is a billion floating-point operations, the basic multiply or add a GPU performs. GPUs are rated in TFLOPS, trillions of operations per second, so dividing GFLOP per token by a GPU's TFLOPS gives milliseconds per token of pure compute.

Why the 2, and not 3 or 4

The 2 is not a fitted constant. It is a count of what the hardware does with each weight. Almost all of a transformer's work is multiplying a vector (the token's current representation) by a weight matrix, and for every single weight in that matrix the hardware does exactly two things: it multiplies the weight by one input number, and it adds that product into a running total. One multiply, one add.

Why 2 operations per weight: one multiply, one add

A tiny layer: 3 inputs, 3 weights, 1 output. Follow any weight: it is touched exactly twice.

input x weight w op 1: multiply op 2: add into running total

$x1 \rightarrow w1 \rightarrow \times \rightarrow +$ $w1 \times x1$ $\rightarrow$ total $+= w1 \times x1$

$x2 \rightarrow w2 \rightarrow \times \rightarrow +$ $w2 \times x2$ $\rightarrow$ total $+= w2 \times x2$

$x3 \rightarrow w3 \rightarrow \times \rightarrow +$ $w3 \times x3$ $\rightarrow$ total $+= w3 \times x3$

output

Tally for this layer

weights used:	3
multiplies:	3
adds:	3

operations: $6 = 2 \times 3$

GPUs do the pair as one fused multiply-add (FMA) instruction.

Scale it up: 70B model = $2 \times 70,000,000,000 = 140$ GFLOP per token. 8B model = 16 GFLOP per token.

Training would use each weight three times (forward, two backward passes) = 6 per weight. Inference is forward only, so it stays at 2.

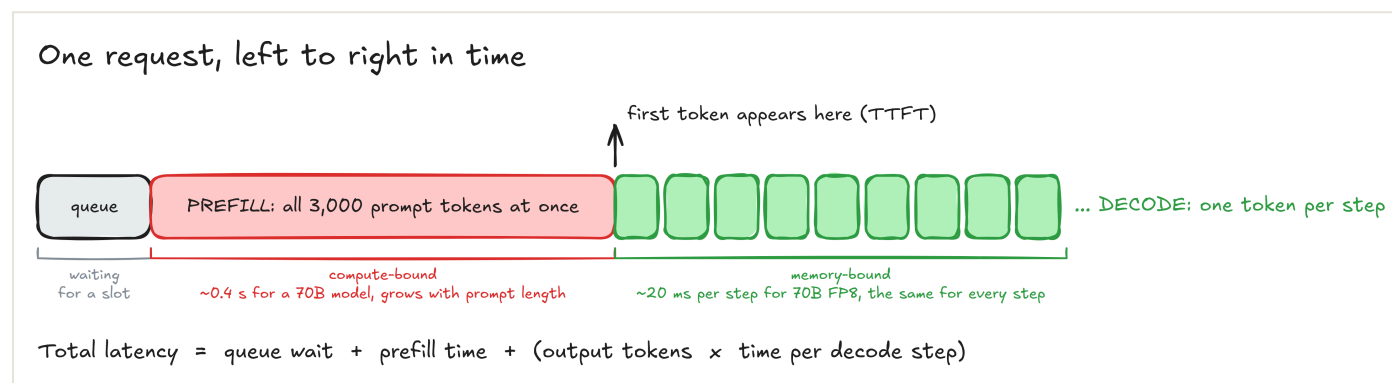
Follow any weight through the diagram: it passes through one multiply and one add, and there is no third operation for it to pass through. Once its product has been folded into the total, the weight's job for this token is done. GPUs even have a single instruction for the pair, the fused multiply-add, which is why "2 per weight" is the natural unit.

The training comparison is what makes the number feel less arbitrary. During training, each weight is used three times per token: once in the forward pass, and twice more going backward to work out how the weight should change. That is 6 per weight, the well-known "6 $\times$ parameters $\times$ tokens" rule for training cost. Inference is the forward pass only, so it stays at 2.

Two things the formula leaves out. Attention's comparison between tokens reads a structure called the KV cache rather than the weights, so it is not counted; for ordinary prompts it adds a few percent, but it grows with context length and matters at very long contexts. And mixture-of-experts models route each token through only some of their weights: DeepSeek V3 stores 671 billion parameters but uses about 37 billion per token, so its cost per token is 2×37 B, not 2×671 B. Use the active parameter count for compute and the total count for memory.

The two phases of a request

Here is the fact the rest of the series rests on. A single request runs in two phases that behave completely differently, and once you see why, most of inference performance becomes predictable.



Prefill is the first phase. The entire prompt, every one of its tokens, is processed in a single forward pass. Because all the prompt tokens are available at once, the GPU can multiply a large block of them against the weights simultaneously, which is the kind of dense, parallel arithmetic GPUs were built for. The limiting factor is how fast the GPU can do arithmetic, so prefill is **compute-bound**. Its duration grows in proportion to the number of prompt tokens: twice the prompt, twice the prefill time. Prefill ends by producing the first output token, so the time a user waits for something to appear (time to first token, or TTFT) is mostly prefill time. Prefill also has a side effect that matters enormously: as it processes each prompt token, it stores that token's attention information in the KV cache, which the next phase reads at every step.

Decode is the second phase. The model now produces the answer one token at a time. Each new token depends on the previous one, so the steps are strictly sequential. Each step is a full forward pass through all the layers, but for only one new token per request. The arithmetic per step is tiny compared to prefill. What is not tiny is that every step still has to read every weight in the model from memory. Reading 70 gigabytes takes time no matter how little math you do with it, so decode is **memory-bandwidth-bound**. Its time per step barely depends on how long the prompt was; it depends on how many bytes of weights there are and how fast the memory can deliver them.

An analogy that holds up: imagine a librarian who must consult every book in a 70,000-volume library to answer any question. When you hand her a long written question, she reads the whole thing and then walks the library once; the reading time grows with the question, and that is prefill. Then she answers you one word at a time, and for every word she must walk the entire library again, because each word might depend on any book. She barely thinks at each shelf; she is limited by walking speed. That is decode. And if ten people are waiting, she can answer one word for each of them per walk at almost no extra cost, because the walk was the expensive part. That last observation is batching, and it is where this article is heading.

Prefill is a few very large matrix multiplications limited by how fast the GPU computes. Decode is hundreds of small ones that each re-read the whole model, limited by how fast memory can be read. Every optimization in this field attacks one of those two limits.

Where the memory goes

The weights

Each weight is stored at some precision, and the precision decides how many bytes it takes:

weight memory = parameters × bytes per parameter

70B in FP16: $70 \times 10^9 \times 2 \text{ bytes} = 140 \text{ GB}$

70B in FP8: $70 \times 10^9 \times 1 \text{ byte} = 70 \text{ GB}$

8B in FP16: $8 \times 10^9 \times 2 \text{ bytes} = 16 \text{ GB}$

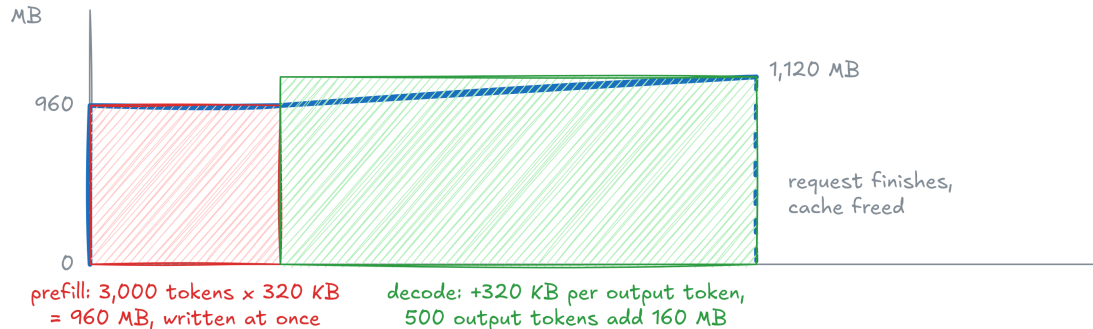
FP16 and BF16 (2 bytes) are the usual "full precision" for serving. FP8 (1 byte) is native on H100 and later and is close to lossless for most models. FP4 (half a byte) is native on Blackwell. Choosing a smaller format is called quantization, and this one multiplication is why it matters: a 70B model in FP16 is 140 GB, which does not fit on an 80 GB H100 at all. In FP8 it is 70 GB, which technically fits, but leaves only 10 GB for everything else, and "everything else" is where the requests live.

The KV cache

Recall that attention lets each token look back at every earlier token. To do that, the model computes, for every token and every layer, two small vectors called the key and the value. When a new token is generated, it compares itself against the keys of all previous tokens and gathers their values. Without a cache, the model would recompute the keys and values of the entire history at every decode step, making step 300 of an answer three hundred times more expensive than step 1. So the engine computes each token's keys and values once, when the token is first processed, and stores them. That store is the KV cache.

Three properties of it matter for serving. It belongs to one request. It grows by one entry per token for the life of the request and is released when the request finishes. And it lives in the same GPU memory as the weights, so every byte of cache is a byte the weights are not using.

KV cache of one request over its lifetime



Llama 3 70B, FP16 cache. One 3.5k-token conversation holds ~1.1 GB of GPU memory for its whole duration.

$\text{KV bytes per token} = 2 \times \text{layers} \times \text{KV heads} \times \text{head dim} \times \text{bytes} = 2 \times 80 \times 8 \times 128 \times 2 = 320 \text{ KB}$

Its size per token is fixed by the model's architecture and can be read off the model card:

$\text{KV bytes per token} = 2 \times \text{layers} \times \text{KV heads} \times \text{head dimension} \times \text{bytes per value}$

Llama 3 70B, FP16 cache: $2 \times 80 \times 8 \times 128 \times 2 = 327,680 \text{ bytes} \approx 320 \text{ KB per token}$

Llama 3 8B, FP16 cache: $2 \times 32 \times 8 \times 128 \times 2 = 131,072 \text{ bytes} \approx 128 \text{ KB per token}$

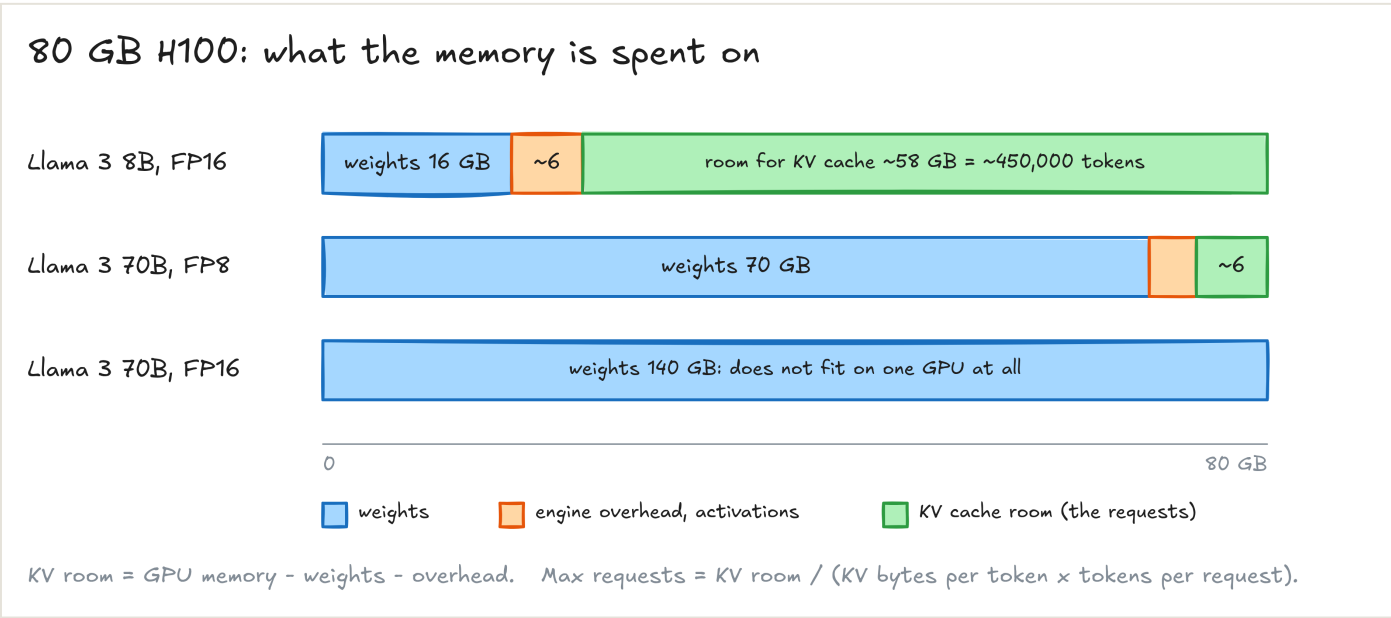
Reading the terms: the 2 is one key vector and one value vector. Layers is how many the model has, and each keeps its own cache. KV heads is the number of attention heads that store keys and values; modern models use grouped-query attention, which shares keys and values across heads, so Llama 3 70B has 64 attention heads but only 8 KV heads. That single design choice cut the cache by 8× and is why long contexts

became affordable. Head dimension is the length of each vector, 128 in the Llama family. Bytes per value is the cache's precision; quantizing the cache to FP8 halves it.

Those per-token figures look small until you multiply them out. A 4,000-token conversation on the 70B model holds $4,000 \times 320$ KB, about 1.3 GB. Sixty such conversations at once need 78 GB, which is an entire H100 with nothing left for the weights.

The memory budget of one replica

This is the calculation to be able to do on a whiteboard. It explains why concurrency has a hard ceiling, why long contexts are expensive even when they are not slow, and what the "memory cliff" in Part 3 actually is. Start with the GPU's capacity, subtract the weights, subtract a working margin for the engine, and what remains is the space for KV cache. Divide that by the cache per token to get how many tokens can be in flight, and divide by the tokens per request to get how many requests.



KV room

= GPU memory - weight memory - engine overhead

tokens in flight

= KV room ÷ KV bytes per token

max requests

= tokens in flight ÷ tokens per request

Llama 3 8B, FP16, one H100:

KV room = 80 - 16 - 6 = 58 GB

tokens in flight = 58 GB ÷ 128 KB ≈ 450,000 tokens

at 1,300 tokens per request (1k prompt + 256 answer + margin): ≈ 350 concurrent requests

at 16,000 tokens per request (a long RAG prompt): ≈ 28 concurrent requests

Llama 3 70B, FP8, one H100:

KV room = 80 - 70 - 4 ≈ 6 GB → ≈ 19,000 tokens → about 6 requests at 3,000 tokens each

Two things to notice. "Tokens per request" is the prompt plus the maximum output you allow, because the engine has to reserve room for the answer to grow; this is why capping the maximum output length frees memory. And the 70B example is why large models are served across two to eight GPUs with tensor parallelism: not only for speed, but to have room for the requests at all.

When someone says "we run this replica at a concurrency of 32," they are implicitly claiming that 32 requests' worth of KV cache fits in memory at this workload's context length. For an 8B model at chat-length prompts, that is comfortably true. For the same model at 16,000-token prompts it is not. If the input distribution has a long tail, the 32 is right for the typical request and wrong for the tail, and the tail is

exactly when memory runs out. The engine responds by refusing new requests, by evicting running ones and recomputing them later, or by crashing. That is the memory cliff, and it is the first thing to check when a deployment falls over under load that a benchmark said it could handle.

Why decode is memory-bound, with numbers

The floor on decode step time

Every decode step reads every weight once. So the fastest a step can possibly be is the time it takes to stream the weights through the memory bus:

```
decode floor = weight bytes ÷ memory bandwidth
```

```
8B FP16 on H100: 16 GB ÷ 3.35 TB/s ≈ 4.8 ms per token
```

```
70B FP8 on H100: 70 GB ÷ 3.35 TB/s ≈ 21 ms per token
```

```
70B FP16 on 8 × H100 (each GPU holds 17.5 GB): 17.5 ÷ 3.35 ≈ 5 ms + communication
```

```
70B FP8 on B200 (~8 TB/s): 70 ÷ 8 ≈ 9 ms
```

Real inter-token latency at batch size 1 sits a little above the floor because of kernel launch overhead and the small amount of arithmetic. If a benchmark reports a number below the floor, something is wrong with the benchmark.

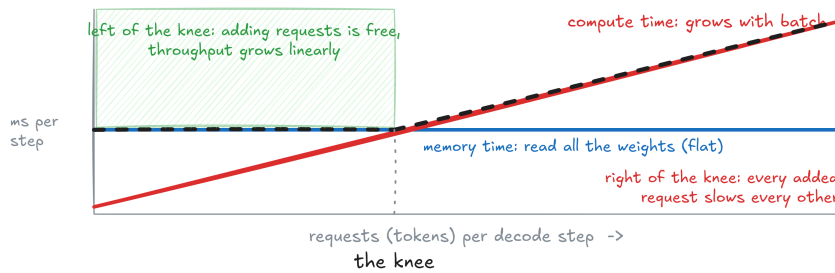
Three practical consequences fall straight out of the division. **Quantization speeds up decode almost proportionally**: FP8 halves the bytes, so it roughly halves the floor, which is why FP8 is close to free performance on H100. **Tensor parallelism speeds up decode**: splitting the weights across GPUs means each reads less per step, at the cost of an exchange of partial results between GPUs at every layer, which is fast over NVLink inside one server and slow across servers. And **faster memory speeds up decode directly**: the jump from H100 to H200 to B200 is, for decode-heavy workloads, mostly a bandwidth story.

Why batching is nearly free, up to a point

If the GPU must read all 16 GB of weights to produce one token for one user, it costs almost nothing extra to produce one token for thirty-two users in the same pass. The weights are read once and applied to thirty-two inputs. The reads were the expensive part; the arithmetic for thirty-two tokens instead of one is small. So at low batch sizes, adding requests barely changes the time per step, and throughput, meaning tokens per second across all requests, grows almost in proportion to the batch.

The question is where this stops. Think of the GPU as having two budgets per step: bytes it can move and arithmetic it can do. Batching adds arithmetic without adding weight reads. While the arithmetic time is smaller than the memory time, the memory time hides it and the step is free. Once the arithmetic time grows past the memory time, the step slows with every added request.

Time for one decode step vs number of requests in the batch



actual step time = $\max(\text{memory time, compute time})$, the dashed line. Crossover on an H100 is ~300 tokens per step in theory; 32 to 128 in practice, because the KV cache is also read every step and grows with batch $\times$ context.

compute time per step = $(2 \times \text{params} \times \text{batch}) \div \text{achieved FLOPS}$
 memory time per step = $(\text{params} \times \text{bytes per param}) \div \text{bandwidth}$
 step time = $\max(\text{compute time, memory time})$

knee, where they cross: $\text{batch} \approx (\text{FLOPS} \div \text{bandwidth}) \times \text{bytes per param} \div 2$
 H100, FP16 weights: $(990 \text{ TFLOPS} \div 3.35 \text{ TB/s}) \times 2 \div 2 \approx 300 \text{ tokens per step}$

The ratio $\text{FLOPS} \div \text{bandwidth}$, about 300 for an H100, is a property of the hardware: it is how many operations the GPU can do per byte it reads. It sets where the knee lands. In an idealized world you could serve about 300 decoding requests on one H100 before compute became the limit. In real benchmarks the knee arrives much earlier, typically between 32 and 128, for two reasons.

Reason one: the KV cache is also read every step

The formula above counted only the weights as bytes read. But attention for each request reads that request's entire KV cache every step, and those bytes grow with both batch size and context length:

KV bytes read per step = $\text{batch} \times \text{context tokens} \times \text{KV bytes per token}$

8B model, batch 64, 4k context: $64 \times 4,000 \times 128 \text{ KB} \approx 33 \text{ GB per step}$ (twice the weights)
 8B model, batch 64, 32k context: $64 \times 32,000 \times 128 \text{ KB} \approx 262 \text{ GB per step} \rightarrow \text{step} \approx 80 \text{ ms}$

This is why long-context workloads have slow decode even at moderate batch: the cache, not the weights, is what the memory bus is delivering. It is also why grouped-query attention was such an important change, and why quantizing the KV cache matters specifically for long context.

Reason two: prefill has the other physics

Prefill on a 3,000-token prompt is a single pass over 3,000 tokens at once, which puts it far past the 300-token knee from the start. It is compute-bound immediately:

prefill time $\approx (\text{input tokens} \times 2 \times \text{params}) \div \text{achieved FLOPS}$

70B, 3,000 tokens, H100 at ~1,000 TFLOPS achieved: $3,000 \times 140 \text{ GFLOP} \div 1,000 \text{ TFLOPS} \approx 0.42 \text{ s}$
 8B, 3,000 tokens: $3,000 \times 16 \text{ GFLOP} \div 1,000 \text{ TFLOPS} \approx 0.05 \text{ s}$
 70B, 30,000 tokens: $\approx 4.2 \text{ s before the first token appears}$

Two things follow. Time to first token is linear in input length, so the tail of the input distribution is the tail of TTFT; a p99 TTFT problem is very often a p99 prompt-length problem. And a long prefill occupies the GPU's arithmetic for hundreds of milliseconds, during which the decode steps of every other request on that replica are delayed. Part 2 covers the three different answers to that problem.

Three numbers to carry with you

Everything above compresses to three formulas. With them you can sanity-check any table anyone shows you.

- **Decode floor** = weight bytes ÷ memory bandwidth. About 21 ms for a 70B model in FP8 on one H100.
- **Prefill time** = input tokens × 2 × parameters ÷ FLOPS. About 0.4 s for 3,000 tokens on a 70B.
- **KV cache per token** ≈ 128 KB for an 8B model, 320 KB for a 70B, in FP16.

Check yourself

Answer aloud before reading the answers.

- 1. Why does time to first token grow with input length while inter-token latency mostly does not?** Because they are set by different phases. TTFT is dominated by prefill, which is compute-bound and processes every prompt token, so it is linear in input length. ITL is one decode step, dominated by reading the weights, and the weights are the same size regardless of the prompt.
- 2. A 70B model in FP16 on a single H100: what happens?** It does not fit. 140 GB of weights against 80 GB of memory. You need at least two GPUs with tensor parallelism just to hold the weights, and realistically four, or FP8 quantization, to leave room for KV cache.
- 3. Batching 32 requests barely slows each one down. Why?** Because the expensive part of a decode step is reading the weights, and that read is the same whether it serves 1 request or 32. The arithmetic for 32 tokens is small relative to the memory time until the batch reaches a few hundred tokens per step.
- 4. What eventually stops batching from being free?** Compute time per step overtakes memory time (the knee); KV cache reads grow with batch × context and pull the knee earlier; and finally memory runs out (the cliff).

Part 2 covers what the serving engine does with this physics: continuous batching, the paged KV cache, and every optimization in the field classified by which limit it attacks.

GPU specifications are NVIDIA's published peak figures, rounded; achieved performance is typically 40 to 70 percent of peak. Worked numbers are rounded for mental arithmetic.

Radx Radhakrishnan works on LLM inference and agentic systems, after a decade of performance work on enterprise data platforms.

Sources

- NVIDIA, H100 Tensor Core GPU: 80 GB HBM3, 3.35 TB/s, 1,979 TFLOPS FP16 with sparsity (989 dense), 900 GB/s NVLink. <https://www.nvidia.com/en-us/data-center/h100/>
- NVIDIA, H200 Tensor Core GPU: 141 GB HBM3e, 4.8 TB/s. <https://www.nvidia.com/en-us/data-center/h200/>
- NVIDIA, DGX B200: 1,440 GB HBM3e and 64 TB/s across 8 GPUs (180 GB and 8 TB/s per GPU). <https://www.nvidia.com/en-us/data-center/dgx-b200/>
- NVIDIA, A100 Tensor Core GPU: 80 GB HBM2e, 2,039 GB/s, 312 TFLOPS FP16 dense. <https://www.nvidia.com/en-us/data-center/a100/>
- Meta, Llama 3 model configuration (70B: 80 layers, hidden size 8,192, 64 attention heads, 8 KV heads; 8B: 32 layers, 4,096, 32 heads, 8 KV heads; vocabulary 128,256). Official repository: <https://huggingface.co/meta-llama/Meta-Llama-3-70B> (gated); an ungated mirror of the same config: <https://huggingface.co/NousResearch/Meta-Llama-3-70B-Instruct/blob/main/config.json>

- Grattafiori et al., "The Llama 3 Herd of Models," 2024. <https://arxiv.org/abs/2407.21783>
- Ainslie et al., "GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints," 2023. <https://arxiv.org/abs/2305.13245>
- DeepSeek-AI, "DeepSeek-V3 Technical Report," 2024 (671B total, 37B activated per token). <https://arxiv.org/abs/2412.19437>
- Kaplan et al., "Scaling Laws for Neural Language Models," 2020, Section 2.1: $C \approx 6N$ per training token, "the factor of 6 accounts for the forward and backward passes." <https://arxiv.org/abs/2001.08361>
- OpenAI Help Center, "What are tokens and how to count them?" (1 token $\approx$ 4 characters $\approx$ 0.75 words). <https://help.openai.com/en/articles/4936856-what-are-tokens-and-how-to-count-them>